

Härte-Track 28.08.2026 — Bonussystem, Automatisierungsketten, Aktendokumente

Ausgangsstand: faa0972 · 2476 node:test · 7378 vitest

Endstand: 2476 node:test · 7468 vitest (+90) · Typecheck 0 · lint:forbidden 0

Live-Verifikation: scripts/verify-bonussystem-live.mjs — 11/11 grün

Wie gesucht wurde

Zwei systematische Durchläufe über die im Auftrag genannten Bereiche, jeweils gegen die Live-Datenbank gegengeprüft (lesend, über das `_run_sql`-Orakel):

1. Alle 411 Route-Handler auf eine Zugangsschranke geprüft. Ergebnis nach Abzug der bewusst öffentlichen Endpunkte (Kontakt, Newsletter, Preisrechner, Health, Tracking): keine ungeschützte Route. `push/send` hat eine Dienstschlüssel-Prüfung, `user/delete/undo` einen 256-Bit-Token, `billing/tariffs/[id]/verifizierung` delegiert an einen Admin-Guard.
 2. Jeder Schreib-Handler gegen die verlangte Berechtigung. Ein Scanner trennte die Handler-Rümpfe und verglich die Berechtigung im Guard gegen die Lese-/Schreib-Einteilung der Rollenmatrix. Sechs Treffer, davon zwei berechtigt (`camt/preflight` und `pilot/camt-dry-run` sind trotz POST nachweislich schreibfrei — 0 Treffer für `insert/update/upsert/delete`) und vier echte: das gesamte Bonusmodul.
-

P0-1 — Ein zweiter Berechnungslauf setzte ausgezahlte Prämien zurück

`fuehreBerechnungslaufDurch()` schrieb je Mitarbeiter:

```
.upsert({ ..., status: 'berechnet', ... },  
{ onConflict: 'regel_id,caregiver_id,zeitraum_von,zeitraum_bis' })
```

Der Unique-Index dahinter existiert live (`bonus_berechnungen_unique`, 28.08. aus `pg_indexes` gelesen) — der Konflikt trat also zuverlässig ein. Ein zweiter Lauf über denselben Zeitraum:

- setzte eine bereits freigegeben oder sogar ausgezahlt markierte Prämie zurück auf berechnet,
- überschrieb `punkte`, `erfuellt` und `messwert` mit neu gerechneten Werten,
- ließ die zugehörige Zeile in `bonus_freigaben` verwaist stehen.

Die Prämie ließ sich danach ein zweites Mal freigegeben — mit einer zweiten Entscheidungszeile zu demselben Vorgang. Bei Status ausgezahlt heißt das: derselbe Betrag ein zweites Mal.

Dieselbe Klasse wie der `monthly_closings`-Upsert (`feddad9`), der einen bereits an den Kostenträger versendeten Monatsabschluss auf `ready` zurückstempelte.

Abhilfe: Der Lauf liest den Bestand des Zeitraums, bevor er schreibt, und überspringt jede Berechnung in einem der drei Endzustände (freigegeben, abgelehnt, ausgezahlt — live aus `bonus_berechnungen_status_check` gelesen). Der Schreibvorgang auf eine vorhandene Zeile ist ein

Compare-and-Swap auf `status='berechnet'`; trifft er ins Leere, wurde parallel entschieden und es wird nichts überschrieben. Ein 23505 beim Anlegen gilt als „parallel angelegt“, nicht als Fehler.

Fail-closed: Lässt sich der Bestand nicht lesen, wird gar nichts geschrieben — ein Lauf, der eine freigegebene Prämie überschreibt, ist schlimmer als ein ausbleibender Lauf.

Übersprungene Berechnungen erscheinen in der Antwort (uebersprungen, gespeichert) und in der Oberfläche als Warnbanner samt Grund je Zeile. Ohne diesen Hinweis sähe ein Lauf, der nichts gespeichert hat, aus wie ein Lauf, der alles neu gerechnet hat.

P0-2 — Freigabe ohne Compare-and-Swap, Nachweis vor dem Statuswechsel

`freigebenBerechnung()` lief in der Reihenfolge: Status lesen → Prüfung `status !== 'berechnet'` → Entscheidungszeile in `bonus_freigaben` schreiben → Status setzen. Ohne CAS.

Zwei gleichzeitige Entscheidungen kamen beide an der Prüfung vorbei, legten beide eine Entscheidungszeile an — möglicherweise gegenläufig, einmal freigegeben, einmal abgelehnt — und schrieben nacheinander den Status. Der letzte gewann. Im Nachweis standen danach zwei Entscheidungen zu einem Vorgang, und welche gilt, war der Zeile nicht anzusehen.

Schlug außerdem das Status-Update fehl, blieb die Entscheidung geschrieben und der Vorgang auf `berechnet` stehen — ein zweiter Versuch legte eine dritte Zeile an.

Abhilfe: Der Statuswechsel läuft zuerst und per CAS (`.eq('status','berechnet')`) — er ist die Beanspruchung des Vorgangs. Die Entscheidungszeile folgt danach; scheitert sie, wird der Status zurückgerollt (gleiche Linie wie `genehmigenAbwesenheit`, `faa0972`). Trifft das CAS ins Leere, unterscheidet die Meldung „gibt es nicht“ von „bereits entschieden (Status: ...)“ — beides als `UserFacingError`, vorher wurde daraus „Interner Serverfehler“.

P1-3 — Abgelehnte Urlaubsanträge kosteten die Prämie

Das Kriterium `keine_ausfaelle` fragte `absences` ab und filterte `**gar nicht nach Status**`. Der Live-CHECK kennt ``beantragt | genehmigt | abgelehnt | storniert`` (nullable, 28.08. gelesen).

Damit zählte ein abgelehnter Urlaubsantrag als Ausfalltag gegen die Prämie: die Kraft hat an dem Tag gearbeitet, *weil* der Antrag abgelehnt wurde — und verlor dafür Geld. Für einen zurückgezogenen (storniert) Antrag galt dasselbe.

Abhilfe: Filter über `abwesenheitBlockiert()` — dieselbe Liste wie in der Einsatzplanung (`BLOCKIERENDE_ABWESENHEITS_STATUS`, `lib/touren/server.ts`) und im DB-Trigger `check_doppelbelegung`. Bewusst keine zweite Wortliste: zwei Vokabulare für dieselbe Frage sind die Fehlerquelle, nicht die Lösung. Die Spalte `status` wird jetzt überhaupt erst mitgelesen — ohne sie wäre kein Filter möglich gewesen.

P1-4 — Prüfhinweise ergaben eine negative Quote

bewerteKeineOffenenPruefungen(gesamt, mitOffenenFehlern, ...) rechnet (gesamt – mitOffenenFehlern) / gesamt und setzt damit eine Zahl von Nachweisen voraus. Gezählt wurden aber Fehlerzeilen:

```
for (const f of fehler) fehlerProCaregiver.set(cg, (...) + 1)
```

review_errors hat live keinen Unique-Index auf service_record_id (nur idx_review_errors_record, 28.08. verifiziert) — mehrere offene Hinweise am selben Nachweis sind ausdrücklich vorgesehen. Zwei Nachweise, davon einer mit drei offenen Hinweisen: $(2 - 3) / 2 = -50\%$. Eine negative Quote, die als messwert in einer numeric-Spalte landete und im Bericht stand.

Abhilfe: Zählung je Nachweis genau einmal (Set<service_record_id>), plus eine Klammer in der Bewertungsfunktion selbst als zweite Linie.

P1-5 — App-Unterschriften zählten nicht

Das Kriterium vollstaendige_dokumentation maß ausschließlich service_records.client_signature. Eine in der App geleistete Unterschrift liegt aber in service_signatures (signer_role='client') — genau so rechnet der Monatsabschluss (lib/abrechnung/monatsabschluss.ts: !r.client_signature && !signedRecordIds.has(r.id)).

Eine Kraft, deren Kundschaft ausschließlich in der App unterschreibt, wurde mit 0 % Dokumentation gemessen und verlor die Prämie.

Abhilfe: Beide Quellen, gebatcht zu 200 IDs wie im Monatsabschluss.

Live-Stand: 30 service_records, davon 26 mit client_signature, 0 Zeilen in service_signatures — noch kein eingetretener Schaden, aber der native Signaturweg ist ausgelieferter Code.

P1-6 — Stornierte Leistungsnachweise zählten gegen die Kraft

Ein Storno schreibt nur proof_status/billing_status = 'STORNIERT'; status bleibt auf signed stehen (kein Storno-Wert im status-Werteset). Der widerrufene Nachweis zählte damit in gesamt und — weil er in aller Regel nicht mehr unterschrieben wird — als fehlende Dokumentation gegen die Kraft.

Abhilfe: ohneStornierte() aus lib/leistungsnachweis/status-sync.ts, dieselbe Quelle wie Rechnungslauf und Monatsabschluss. proof_status und billing_status stehen jetzt im select — ohne sie sind beide undefined und der Filter entfernt nichts.

P1-7 — Drei verschiedene Antworten auf „wer darf Boni verwalten“

Stelle	Regel	Rollen
Oberfläche /admin/bonuses	personal.lesen / personal.schreiben	admin, pdl
Schnittstelle /api/admin/analytics/bonuses	berichte.lesen	admin, pdl, qm, buchhaltung

Die Datenbank gilt — live aus pg_policies gelesen: alle sechs Policies auf den drei Tabellen tragen USING is_admin() und WITH CHECK is_admin(), und is_admin() ist auf ARRAY['admin','superadmin'] beschränkt (aus pg_proc.prosrc).

Der Unterschied gab den Rollen dazwischen keinen zusätzlichen Zugriff — aber eine falsche Auskunft, und das ist die eigentliche Schadenslage:

- Lesewege (GET /regeln, /historie) laufen über den RLS-Client. RLS filtert zeilenweise, ohne Fehler. Die PDL bekam eine leere Regelliste und eine leere Freigabeliste — und keinen Hinweis darauf, dass sie etwas nicht sehen darf. Genau die stillen 0/leer-Werte der QM/PDL-Dashboards (d707cda) und des Angehörigenportals (48d6f3b).
- Schreibwege bekamen 42501, das der Fehler-Sanitizer zu „Interner Serverfehler“ verkürzt: die Anwendung sah kaputt aus, wo sie nur nein sagte.

Besonders unpassend war berichte.lesen bei QM: die Rolle ist ausdrücklich so geschnitten, dass sie die geprüften Daten nicht ändert („sonst prüfte es die eigene Korrektur“) — und die Bonuskriterien speisen sich aus genau diesen Prüfdaten (review_errors).

Abhilfe: Neue Berechtigung bonus.verwalten unter dem Vorbehalt der Administration (NUR_ADMINISTRATION). Sie sagt dasselbe wie is_admin() und wird vor der Datenbank geprüft — die Antwort ist ein ehrliches 403 statt einer leeren Liste oder eines erfundenen Serverfehlers. Oberfläche, Schnittstelle und der längere Bereichs-Präfix /api/admin/analytics/bonuses tragen jetzt dieselbe Regel.

Bewusst keine neue RLS-Policy: die Datenbank sagt bereits das Richtige. Zu ändern war die Schnittstelle, die ihr vorauslief.

P2 — Weitere Befunde im Bonusmodul

- Zeitraum ungeprüft. zeitraum_bis >= zeitraum_von ist live ein CHECK; die verdrehte Eingabe kam als 23514 und damit als „Interner Serverfehler“. Ein nicht lesbares Datum machte aus new Date(von).getTime() NaN → Math.max(1, NaN) = NaN → NaN <= schwellenwert ist false: die Prämie fiel still aus, ohne dass irgendwo ein Fehler sichtbar wurde, und messwert: NaN ging in eine numeric-Spalte. assertZeitraum() prüft jetzt Format, Kalendertag (per Rückrechnung — Date.parse('2026-02-30') ist nicht verlässlich NaN) und Reihenfolge.
- Regelwerte ungeprüft. schwellenwert hat keinen CHECK; NaN oder ein negativer Wert ließen sich anlegen und stellten danach jeden Lauf still auf „nicht erfüllt“. Name, Punkte und Kriterium ebenfalls als UserFacingError mit Nennung der zulässigen Werte.
- Oberfläche wertete Antworten nicht aus. entscheiden() — die Freigabe-Aktion — ignorierte die Antwort vollständig (await fetch(...) ohne jede Prüfung). Scheiterte die Freigabe, lud die Seite die Liste nach, alles sah unverändert aus, und es gab keinerlei Hinweis. regelToggle() genauso; loadBerechnungen() ließ die Liste bei einem Ladefehler einfach leer stehen — von „keine Berechnungen“ nicht zu unterscheiden.
- Nackte Error. „Berechnung ist bereits entschieden (Status: ...)“, „Regel nicht gefunden“ — beides Auskünfte, die der Bedienung weiterhelfen, und beides vom Sanitizer zu „Interner Serverfehler“ verkürzt. Jetzt UserFacingError.

P2-8 — Die Automatisierungsketten erreichten die PDL nie

Sämtliche Ketten in lib/automation riefen ['admin','superadmin'] — die Rolle pdl war nirgends Empfängerin, obwohl die Variablen pdlId bzw. pdlIds heißen, die Kommentare „an PDL/Admin“ sagen und eine ganze Kette vitalwerte-pdl.ts heißt.

Betroffen: Fristablauf, fehlender Nachweis, Budgetgrenze, kritischer Vitalwert, Monatsabschluss-Prüfung, Abrechnungsfehler-Aufgabe. In einer Organisation mit Pflegedienstleitung, aber ohne im Tagesbetrieb tätige Administration gingen diese Meldungen ins Leere. Zusätzlich landete jede automatisch erzeugte Aufgabe bei der Administration, auch wo eine PDL vorhanden war.

Abhilfe: BETRIEBS_EMPFAENGER_ROLLEN = ['admin','superadmin','pdl']; ersterPdlDerOrg() bevorzugt die PDL und fällt erst dann auf die Administration zurück. qm steht bewusst nicht dabei: das Qualitätsmanagement prüft, es disponiert nicht.

Live-Stand: Es existieren derzeit keine pdl-, qm- oder buchhaltung-Konten (Rollenverteilung 28.08.: 3 superadmin, 1 admin, 20 engel, 5 fahrer, 34 kunde). Kein eingetretener Schaden — der Befund wird an dem Tag scharf, an dem die erste PDL eingerichtet wird.

P2-9 — Ein gesperrtes Aktendokument blieb herunterladbar

GET /api/akten/dokumente/[id]/download prüfte allein auf 'archiviert'. Der Live-CHECK auf akten_dokumente.status kennt fünf Werte: entwurf | aktiv | archiviert | gesperrt | abgelaufen. Ein Dokument, das ausdrücklich auf gesperrt gesetzt wurde, bekam weiterhin eine signierte URL — die Statusauswahl der Akte war an dieser Stelle reine Anzeige.

Abhilfe: DOKUMENT_STATUS_OHNE_AUSLIEFERUNG an einer Stelle (lib/akten/types.ts), fail-closed gegen unbekannte Werte. 'abgelaufen' bleibt bewusst auslieferbar: eine abgelaufene Genehmigung ist nicht mehr gültig, aber weiterhin einsehbar — darauf beruht die Ablaufwarnung, die zum Nachfordern auffordert.

Vom booleschen Kennzeichen gesperrt ist das getrennt: das bedeutet in diesem Modul Beweissicherung (Schreibsperre gegen Bearbeiten, Löschen und Versionieren) und lässt das Lesen für die Verwaltung ausdrücklich zu. Für Dritte gilt das nicht — im Angehörigenportal sperrt es auch das Lesen (48d6f3b).

Testbug (kein Produktionsfehler): rote Suite auf main

lib/personal/__tests__/pruefe-budget.test.ts schlug bereits **auf dem unveränderten HEAD** fehl (per git stash gegengeprüft). Der Hilfsfunktion tagVerschoben() lag new Date().toISOString() zugrunde, also das UTC-Datum, während pruefeBudget() gegen heuteBerlin() vergleicht. Im Sommer liegt Berlin zwei Stunden vor UTC: zwischen 00:00 und 02:00 Berliner Zeit lieferte tagVerschoben(0) gestern, der Übertrag galt als verfallen, und der Test „Übertrag gilt am Verfallstag selbst noch“ meldete 100 statt 50.

Nur der Test war falsch — die Regel heute > verfaelltAm ? 0 : uebertrag ist

richtig und ist unverändert. Helfer und der Halbjahres-Vergleich rechnen jetzt gegen die Berliner Uhr.

Migration

20261014000000_rollenmatrix_bonus_verwalten.sql (+ Rollback ...0001) ergänzt public.rollen_matrix() um bonus.verwalten für admin/superadmin.

Nicht Voraussetzung für die Wirksamkeit der Härtung. Die bonus_*-Policies bleiben unangetastet auf is_admin() und sagen bereits dasselbe; der Anwendungs-Guard prüft gegen die TypeScript-Matrix. Die Migration hält nur den SQL-Spiegel vollständig, damit eine künftige Policy public.darf(...) benutzen kann, ohne fail-closed ins Leere zu laufen.

Nicht live eingespielt: In dieser Sitzung stand kein Supabase-MCP zur Verfügung, und ein Apply per service_role scheitert grundsätzlich an fehlenden DDL-Rechten (42501). Der PGLite-Test `__tests__/security/rollenkonzept-pglite.test.ts` spielt die Migration gegen ein echtes Postgres ein und hält den Gleichstand SQL ↔ TypeScript.

Prüfläufe

Lauf	Ergebnis
npm vitest run	7468 bestanden, 38 übersprungen, 0 rot (vorher 7378)
npm run test:unit (node:test)	2476 bestanden, 0 rot
npm run typecheck	0 Fehler
npm run lint:forbidden	0 verbotene Strings (24 754 Dateien)
node scripts/verify-bonussystem-live.mjs	11/11 grün

90 neue Tests in drei Dateien, jeweils mit Gegenprobe:

- `__tests__/analytics/bonusEngine-haertung.test.ts` (52) — Berechnungslauf, Freigabe-CAS, Messwertermittlung, Zeitraum, Regelwerte, Mandanten-Fence, plus vier Gegenproben, die die alten Rechnungen noch einmal ausführen (–50 % Quote, 5 Ausfalltage aus einem abgelehnten Antrag, 0 % bei App-Unterschrift, NaN-Zeitraum als stilles „nicht erfüllt“).
- `__tests__/automation/org-empfaenger.test.ts` (14)
- `__tests__/akten/dokument-auslieferung.test.ts` (11)

Vier bestehende Wächter-Tests schlugen durch die Modelländerung an und wurden bewusst nachgezogen — genau dafür sind sie da: `rollenkonzept.test.ts` (Vorbehaltsliste), `rollenkonzept-pglite.test.ts` (Migration einspielen), `admin-api-routen-guard.test.ts` (neuer Helfer in GUARDS), `rollen-tenant-crossover-pglite.test.ts` (Berechtigung ohne Zieltabelle) sowie die Migrationsliste in `pre-pilot-snapshot.ts`.

Offen / bewusst nicht getan

- Kein Backfill. Alle drei bonus_*-Tabellen sind live leer (0 Zeilen). Es gibt keinen Altbestand zu korrigieren; die Prüfungen G:* im

Verifikationsskript würden ihn finden.

- bonus_*-RLS unverändert. is_admin() ist bereits richtig.
- requireOpsAdmin liest nur profiles.role, nicht app_metadata.role — anders als holeRolle() in lib/auth/guard.ts, wo app_metadata Vorrang hat. Fail-closed in der einen Richtung (eine nur in app_metadata gesetzte Rolle gilt nicht), aber eine über app_metadata herabgesetzte Rolle wirkt auf Ops-Routen nicht. Notiert als eigener Befund, nicht in diesem Track angefasst — er betrifft alle 48 Routen mit diesem Guard.